

```

output "asg_max_size" {
  description = "asg max size"
  value = "1"
}

output "asg_min_size" {
  description = "asg min size"
  value = "3"
}

output "asg_protect_from_scale_in" {
  description = "Allows setting instance protection."
  value = "true"
}

//asg lifecycle hook default vars
output "asg_lifecycle_hook_default_result" {
  description = "asg lifecycle hook default result"
  value = "CONTINUE"
}

output "asg_lifecycle_hook_heartbeat_timeout_when_scale_out" {
  description = "asg lifecycle hook heartbeat timeout"
  value = "30"
}

output "asg_lifecycle_hook_heartbeat_timeout_when_scale_in" {
  description = "asg lifecycle hook heartbeat timeout"
  value = "60"
}

```

Figure 5: Terraform variables

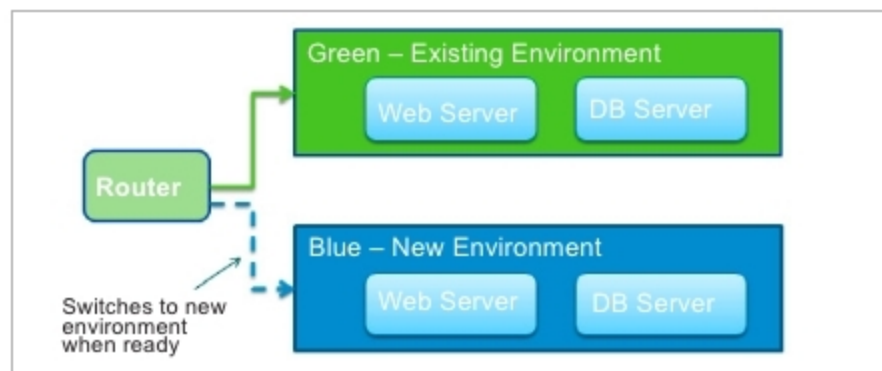


Figure 6: Blue-Green deployment

CI/CD pipeline workflow

The CI/CD pipeline workflow for applying changes to the infrastructure using Terraform is given below.

1. The developer or operations engineers change the Terraform configuration file in their local machine and commit the code to BitBucket.
2. GitBucket WebHook triggers a continuous integration job to Jenkins.
3. Jenkins will pull the latest code from the configured repo, which contains Terraform files to its workspace.
4. It reads the Terraform configuration and then initialises the remote Consul back-end.
5. Terraform generates a plan about the changes that have to be applied on the infrastructure.
6. Jenkins sends notifications to a Slack channel about the changes, for manual approval.
7. Here, the user can approve or disapprove the Terraform plan.
8. The user input is sent to the Jenkins server for further action.
9. Once the changes are approved by an operator, Jenkins

```

module "service1" {
  health_check_grace_period = "${var.asg_health_check_grace_period}"
  health_check_type         = "${var.asg_health_check_type}"
  launch_configuration       = "auto-scale-service1"
  max_size                  = 2
  min_size                  = 4
  name                      = "service1"
  vpc_zone_identifier        = ["subnet-aa467431", "subnet-7f573793"]

  tag {
    key   = "Name"
    value = "service1"
    propagate_at_launch = "${var.asg_propagate_at_launch}"
  }

  tag {
    key   = "env"
    value = "stage"
    propagate_at_launch = "${var.asg_propagate_at_launch}"
  }
}

```

Figure 7: Calling Terraform modules from the workspace

will execute the Terraform Apply command to reflect the changes to the infrastructure.

10. Terraform will create a report about the resources and the dependencies created while executing the plan.
11. Terraform will provision the resources in the provider environment.
12. Jenkins will again send a notification to the Slack channel about the status of the infrastructure after applying the changes to it. Once the job is executed, the Jenkins pipeline job is configured to clean up the workspace created by the job.

Setting up the deployment environment

1. Create a repo in scm tools like GitLab or BitBucket, and commit the Terraform configuration and its dependency module to the repo. If you are using any third party remote module as a dependency, it will be automatically downloaded during execution.
2. If you do not have the Jenkins server, then just pull a Jenkins Docker image and run it on your local machine. If you are setting it up in a cloud environment, check the Jenkins AMI (Amazon Machine Image) from the marketplace to set up the environment and configure the required plugins.
3. Create a WebHook in your BitBucket repo settings to invoke a *http* call to the Jenkins call back URL for

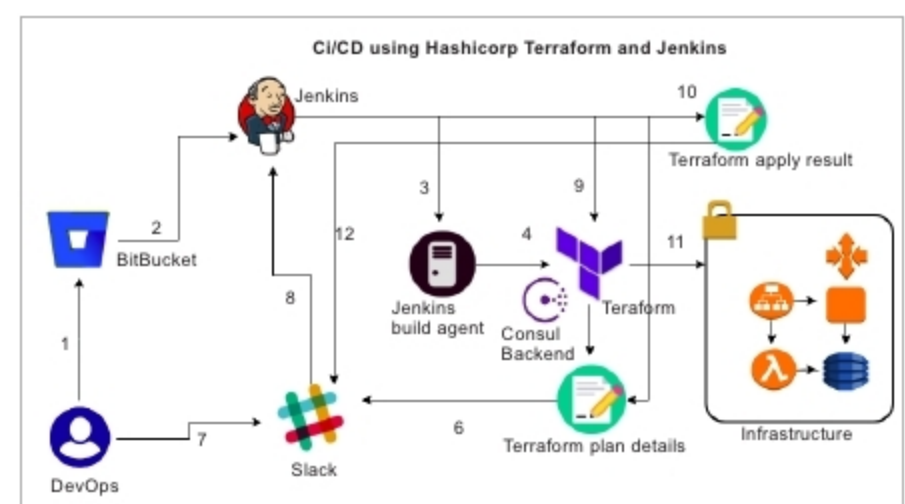


Figure 8: CI/CD using HashiCorp Terraform and Jenkins